Complete Implementation Plan

Secure and Privacy-Preserving Federated Deep Learning Training Platform for Medical Imaging

Results are experimental, on public datasets, and are not clinical performance.

1. Implementation Vision

The project is a complete software platform rather than only a collection of machine-learning scripts. It serves three separate roles: **Admin**, **Researcher** and **Hospital Operator**. Today all three use one web application, which shows each role its own screens. Packaging it as desktop applications and adding a companion mobile application are still planned. Advanced functions such as dataset management, preprocessing, local training, detailed configuration, model management and inference stay on the desktop; every mobile function must also exist on the desktop.

The core privacy principle is that raw medical images remain inside the hospital environment. Hospitals perform local training and exchange only model updates (with differential privacy when the job enables it) and required telemetry with the federated infrastructure. Federated learning alone does not make the system private; the system uses a central server, so it is federated, not peer-to-peer.

Roles

Role	Account	Can do	Cannot do
Admin (admin)	Cannot self-register. Created through a controlled mechanism such as a seed script.	Approve, reject and suspend users. View audit logs and update validation. Create federation jobs (strategy, DP and security settings). Approve hospital join requests. Start and stop Flower runs, open and aggregate rounds. Evaluate and promote global models.	Access hospital datasets or raw images.
Researcher (researcher)	Registers, then stays pending until an Admin approves.	Publish training requests (disease, task, architecture, epochs, batch size). Follow each participating hospital's local training progress live.	Create or run federation jobs, see the global model registry, approve users, or access hospital data. Researcher access to federation results is still to be designed.
Hospital Operator (hospital_operator)	Registers against an existing active hospital. Tied to one hospital_id.	Inspect and preprocess local datasets, train locally, run local inference. Join training requests, request to join federation jobs, and receive round notifications.	See another hospital's data, or use Admin and Researcher functions.

2. Final Software Architecture

Layer	Main Components	Responsibilities
User Applications	One React + Vite web app with Admin, Researcher and Hospital screens. Desktop packaging and mobile are planned.	Configuration, monitoring, dataset and training controls, model and security views, communication.
Application Backend	Node.js / Express, REST APIs, Socket.io	Authentication, roles, audit, training requests, federation routes, notifications. Starts one Flower server per federation job.
Data Layer	MongoDB; SQLite (federation_metadata.db); model files on disk	MongoDB: users, hospitals, audit logs, training requests, notifications. SQLite: jobs, rounds, updates, global models, evaluations, DP ledger. Large model files are not stored in a database.
Federated Learning	Python + Flower (federation/)	Jobs, rounds, update validation and security checks, FedAvg, FedProx, coordinate-wise median, trimmed mean, global models, evaluation and promotion.
Deep Learning	PyTorch; 8 architectures in one registry	Model definition, one trainer for every architecture, validation and inference.
Hospital Client	Local Python pipeline (hospital_client/) and local ML service on 127.0.0.1:8765	Private dataset handling, preprocessing, resource monitoring, local training, DP, Flower client, local inference.

3. Repository and Project Foundation

Actual structure: frontend, backend, federation, hospital_client, scripts, docs (one document per module) and evaluation_datasets. Tests sit with the code they check. Status: **Implemented**

- Git/GitHub repository with one branch per module. **To do:** the default branches stop before Module 9, so Modules 9 to 12 still need merging into one integration branch.
- Separate environments for frontend, backend and Python components.
- Configuration through environment variables and configuration files, with secrets outside source code.
- Common identifiers for users, hospitals, jobs, rounds and model versions.
- Logging and error-handling conventions, plus unit and integration test directories.

4. Module-Wise Implementation Plan

Implemented = working code with automated tests. **Partial** = some of the module exists. **Planned** = not built yet.

#	Module	Status	#	Module	Status
1	Authentication and Roles	**Implemented**	12	Byzantine Detection	**Implemented**
2	Admin and Researcher App	**Partial**	13	Monitoring and Telemetry	**Partial**
3	Hospital App	**Implemented**	14	Database, Audit, Storage	**Partial**
4	Dataset Inspection	**Implemented**	15	Model Versioning	**Partial**
5	Preprocessing	**Implemented**	16	Local Inference	**Implemented**
6	Model Management	**Implemented**	17	Mobile Application	**Planned**
7	Local Training	**Implemented**	18	Communication, Notification	**Partial**
8	Resource-Aware Training	**Implemented**	19	Testing and Reliability	**Partial**
9	Federated Learning Engine	**Implemented**	20	Research Experiments	**Partial**
10	Differential Privacy	**Implemented**	21	Packaging and Deployment	**Planned**
11	Secure Communication	**In progress**			

Module 1: Authentication and Role Management

Status: **Implemented**

Done

- Registration, login, logout, password change and JWT authentication, with failed-login lockout.
- Three separate roles (Admin, Researcher, Hospital Operator) enforced on API routes and application screens.
- Admin approval for researchers; admin accounts cannot be created through public registration.
- Hospital users tied to a hospital_id, with hospital isolation.
- Authentication and authorization events recorded in audit logs.

To do

- Manual penetration testing and input fuzzing as project-level hardening.

Module 2: Researcher/Admin Application

Status: **Partial**

Done

- Admin screens: dashboard, approvals, hospitals, federation jobs and rounds, global models, update validation and security, users, audit log.
- Researcher screens: dashboard, training requests, live monitoring, hospitals, profile and settings.
- Job configuration: rounds, epochs, learning rate, batch size, strategy, DP and security thresholds, with validation.
- Start and stop of Flower runs, and aggregate-now once enough updates have arrived.

To do

- Replace the mock Experiments, Notifications and Messages screens, and the Halt, Export and Quarantine dialogs.
- Decide which federation results researchers may see.
- Ensure every mobile function is also on the desktop once mobile exists.

Module 3: Hospital Application

Status: **Implemented**

Done

- Local dataset selection, inspection results and preprocessing status, hiding Python/Flower complexity.
- Hardware-based training recommendations, local training with live curves, results and resource statistics.
- Local model registry, local inference, federation status, automatic training during Flower rounds.
- Real round notifications.

To do

- Replace mock operator messages (Module 18).
- Show and deploy the promoted global model (Modules 15 and 16).

Module 4: Dataset Ingestion and Inspection

Status: **Implemented**

Done

- Image folders plus CSV, XLS and XLSX; labels from class folders or a metadata CSV.
- Image readability, dimensions and channels; corrupted and invalid files; SHA-256 duplicates.
- Sample counts, class distributions, imbalance warnings and existing split detection.
- dataset_profile.json and dataset_report.md, with counts and names only (no pixels, no rows).

To do

- Optional: existing-split contract for tabular datasets.

Module 5: Automated Preprocessing Engine

Status: **Implemented**

Done

- Validation, quarantine with nothing deleted, and full sample accounting.

• Reproducible stratified, grouped (patient-level) or existing splits, with leakage checks.
• Lazy manifests by default, or a separate materialized copy. Original files are never changed.
• Inverse-frequency class weights from the training split only; tabular statistics fitted on training data only.
• Resizing and channel handling are consistent across clients (applied through Module 7). Images are scaled 0-1.

To do

• Apply oversampling and undersampling: ratios are recorded but no samples are repeated or dropped.
• Mean/std normalisation is not implemented.
• Expose the imbalance strategy in the CLI and app (Python configuration only today).

Module 6: Model Management

Status: **Implemented**

Done

• One registry with 8 architectures: ViT-B/16, EfficientNet-B4, EfficientNet-B0, ResNet-50, ResNet-18, MobileNetV2, MobileNetV3-Small, MobileViT-XXS.
• No single fixed model: Module 8 recommends one per machine, and EfficientNet-B0 is no longer the forced primary model.
• Immutable versioned safetensors checkpoints, with SHA-256, architecture and layer-shape checks on every load.
• Configuration separate from code; adding a model takes one factory function and one registry entry.

To do

• Pretrained weights work from a local copy only and are not offered in the app, which trains from scratch.

Module 7: Local Deep Learning Training

Status: **Implemented**

Done

• One PyTorch trainer for all architectures: checkpoints, resume, early stopping, schedulers, mixed precision.
• Accuracy, macro precision/recall/F1, per-class metrics, confusion matrix, ROC-AUC, plots.
• Best checkpoint chosen on validation loss; the test split is evaluated once at the end.
• FederationHandoff contains parameters and metadata only. FedProx proximal term and DP-SGD are supported.

To do

• Expose class weighting in the app (CLI only today). ISIC 2019 unweighted reached 85.08% accuracy but 24.7% melanoma recall.
• Evaluate class-weighted training.

Module 8: Resource-Aware Training

Status: **Implemented**

Done

• Detects GPU, free VRAM, GPU load, CPU, RAM and disk, with a short network availability and latency probe.
• Per-model memory and time estimates using a real one-step dry run; Fast, Recommended, High-Capacity and Manual options.
• Runtime monitoring and resource statistics; on CUDA out-of-memory, retries with half the batch size, at most twice.
• Policy thresholds are configurable. Tested on the RTX 3050 (4 GB) and on simulated hardware profiles.

To do

• Test on real machines with different hardware.
• Dry-run time estimates are optimistic when image loading is slow; adaptation happens per run, not mid-epoch.

Module 9: Federated Learning Engine

Status: **Implemented**

Done

- One Flower server per job, started from the Admin screen, plus a manual-round protocol and a Flower simulation bench.
- Join requests with Admin approval, base-model distribution with checksum, update validation.
- Sample-weighted FedAvg, FedProx (API and bench), round plan with early stopping, BatchNorm calibration on Flower jobs.
- Global model versions, held-out evaluation and Admin promotion.
- Real runs on malaria data: 2 hospitals, 2 rounds, ResNet-18, 96.14% on the held-out set (one seed; held out per image, not per patient).

To do

- Run across separate machines: every run so far used one machine. Manual-round jobs still submit a folder path, so they need a shared machine.
- Add BatchNorm calibration for manual-round jobs.
- Flower's start_server/start_client calls are deprecated and should be replaced eventually.

Module 10: Differential Privacy

Status: **Implemented**

Done

- Opacus DP-SGD with Poisson sampling, per-sample clipping and calibrated Gaussian noise, set per job.
- RDP accounting over all rounds; budget checks before training; server recomputes epsilon and enforces the cap through a ledger.
- Real run: MobileNetV3-Small, 2 rounds, 95.67% accuracy, epsilon 3.147 and 4.909 for the two hospitals at delta 1e-5.

To do

- Controlled comparison with a same-architecture non-DP job. The current ResNet-18 comparison is indicative only.
- Known limits: the guarantee is per image, not per patient; the server cannot verify that noise was added; DP cannot be combined with FedProx or fp16.
- Optional: membership-inference audit and patient-level privacy.

Module 11: Secure Communication

Status: **In progress**

Done

- Backend HTTPS and Socket.io over WSS (TLS 1.2 or newer), and Flower gRPC over TLS with certificate verification.
- Hospital ML service verifies the backend certificate; no silent downgrade to plain connections.
- Development CA script; secrets stay outside source code; each channel is documented.
- Hospitals are authenticated by JWT over TLS. Browser to local ML service stays on loopback HTTP by design.
- Automated tests pass. Manually confirmed so far: HTTPS API without a certificate warning, and WSS socket connection.

To do

- Finish the browser checks and a complete federation run over TLS, then mark the module completed.
- Restrict CORS and Socket.io origins for deployment (any origin is accepted today).
- Future: mutual TLS, HSTS, automated certificate issuance and revocation checks.

Module 12: Byzantine / Malicious Update Detection

Status: **Implemented**

Done

- Intake checks before aggregation: NaN/Inf, sample-count consistency, L2 delta norm, cosine direction (from round 3), metric plausibility.
- ACCEPT, FLAG, QUARANTINE and REJECT verdicts, recorded and shown to the Admin. Thresholds are configurable per job.
- Robust aggregation: coordinate-wise median and trimmed mean.
- Reproducible benchmark (federation/eval_m12.py, synthetic updates): quarantine reduced error 90.8% for 12x scaling and 98.9% for noise.

To do

- Add a real release-from-quarantine action; the Quarantine dialog is mock.
- Known limits: robust aggregation needs at least 3 hospitals; sign-flip is undetected in rounds 1 and 2; FLAG only alerts; skipped checks do not block; adaptive attacks inside the bounds are not covered.
- Run attack experiments on real training (Module 20). Future: Krum and adaptive thresholds.

Module 13: Monitoring and Telemetry

Status: **Partial**

Done

- Hospital: live training curves, logs and measured local hardware.
- Researcher: live per-hospital epoch, loss and accuracy through Socket.io.
- Admin: live Flower panel with round, participants and each hospital's current step; update validation results.

To do

- Send per-hospital CPU, RAM, GPU and VRAM telemetry to the Admin (the screen reports it as not available).
- Add historical charts, and expose status to mobile.

Module 14: Database, Audit and Model Storage

Status: **Partial**

Done

- MongoDB: users, hospitals, audit logs, training requests, notifications.
- SQLite: federation jobs, rounds, updates, global models, evaluations, promotion decisions, DP ledger.
- Model files on disk, with metadata and references in the databases. Raw medical images are never stored centrally.

To do

- Record Admin federation actions in the audit log (authentication, admin and training-request events are recorded today).
- Store round and resource metrics for history.
- Decide whether federation metadata stays in SQLite or moves to MongoDB.

Module 15: Model Versioning

Status: **Partial**

Done

- Immutable local versions (Module 6) and global model versions linked to job, round, participants and evaluation (Module 9).
- Admin promotion to MAIN per model track, never automatic. Versions and status shown on the Admin screen.

To do

- Let hospitals download the promoted MAIN model.
- Add platform-wide approval, rollback to an earlier version, and model export.

Module 16: Local Inference

Status: **Implemented**

Done

- Prediction on one new local image with an explicitly chosen model version; checks hash, shapes and class mapping.
- Applies the same deterministic preprocessing as validation and test. The image is not uploaded.
- Outputs predicted class, probabilities and model version, as a model prediction rather than a diagnosis.

To do

- Run promoted global models: only locally stored models work today, and results report platform_approved = false. This needs Module 15.
- Known limits: DICOM and 16-bit images are not supported; confidence is uncalibrated; no out-of-distribution detection.

Module 17: Mobile Application

Status: **Planned**

To do

- Login and role-based access through the same backend, for all three roles as appropriate.
- Training status, round progress, hospital status, model status and key metrics.
- Security alerts, notifications and communication.
- Only administrative actions that are safe on mobile; every one mirrored on the desktop.

Module 18: Communication and Notification

Status: **Partial**

Done

- Hospital notifications for round opened and closed, stored in MongoDB and delivered live through Socket.io.

To do

- Researcher-to-hospital and hospital-to-researcher messaging (mock today).
- Admin and Researcher notifications (mock today).
- Training completion and failure alerts, connection errors, and security alerts for suspicious updates.

Module 19: Testing and Reliability

Status: **Partial**

Done

- Last full run on 23 October 2026: federation 142 passed; hospital_client 865 passed, 1 skipped; backend 188 passed; frontend build OK.
- Coverage: preprocessing, models, training, aggregation, privacy, TLS negative cases, malicious updates, authentication.
- End-to-end Flower test with two hospital services.

To do

- Multi-machine runs, client dropout, interrupted-round recovery and Non-IID tests on real data.
- Frontend automated tests (none today), and full end-to-end tests across all three roles.

Module 20: Research Experiments and Evaluation

Status: **Partial**

Done

- 8-model malaria benchmark on one test set (2,799 images, pretrained, Colab Tesla T4, run outside the app): EfficientNet-B4 98.04%, ResNet-18 98.00%, ResNet-50 97.11%, ViT-B/16 97.32%. One run per model; 95% intervals overlap.
- Flower bench (3 IID clients, 5 rounds, MobileNetV2 from scratch): 97.2% on the same test split.
- Byzantine benchmark on synthetic updates; indicative DP comparison.

To do

- Experiment 1: centralized baseline versus federated training (controlled).
- Experiment 2: FedAvg versus FedProx under Non-IID data.
- Experiment 3: federated training without DP versus with DP (same architecture).
- Experiment 4: system without Byzantine defense versus with it, under real training.
- Experiment 5: fixed resource configuration versus resource-aware training.
- Experiment 6: communication cost, training time and round efficiency.
- Repeat runs with several seeds and replace the mock Experiments screen.
- Record configuration, seeds, dataset partitioning and results for reproducibility.

Module 21: Packaging and Deployment

Status: **Planned**

To do

- Package the Admin/Researcher application with a desktop wrapper such as Electron or Tauri.
- Package the Hospital client so users do not need to run Python commands manually.
- Build the mobile application for supported Android/iOS targets as applicable.
- Create deployment configuration, production certificates and setup instructions.
- Test installation, startup, update and connection behavior on clean machines.

5. Development Sequence

Phase	Work	Status
1	Repository, environments, configuration and coding standards	**Done**
2	Node.js backend, database schema, authentication and basic APIs	**Done**
3	Admin and Researcher application shell and dashboards	**Done**
4	Hospital application shell and registration workflow	**Done**
5	Dataset ingestion, validation and dataset reporting	**Done**
6	Preprocessing engine and reproducible local data pipeline	**Done**
7	PyTorch model implementation and standalone local training	**Done**
8	Multi-client Flower federation simulation	**Done**
9	FedAvg baseline and round management	**Done**
10	Real-time monitoring and telemetry	**Partial**
11	Resource-aware training	**Done**
12	FedProx (implemented) and Non-IID experiments (not yet run)	**Partial**
13	Differential Privacy	**Done**
14	TLS, authentication hardening and secure communication	**In progress**
15	Byzantine/outlier detection and attack benchmark	**Done**
16	Model versioning, checkpoint management and approved-model workflow	**Partial**
17	Local inference	**Done**
18	Mobile application and feature parity	**Planned**
19	Full integration, reliability and security testing	**Partial**
20	Research experiments, result collection and evaluation	**Partial**
21	Desktop/mobile packaging, deployment and final demonstration	**Planned**

Recommended order for the remaining work

1. Finish the Module 11 checks and merge Modules 9 to 12 into one integration branch.

2. Deliver the promoted global model to hospitals and run it in local inference (Modules 15 and 16).
3. Run a federation across separate machines over TLS.
4. Complete Module 13 telemetry and Module 18 messaging and notifications, removing the mock screens.
5. Run the Module 20 experiments and write up the results.
6. Build the mobile application (Module 17).
7. Complete integration testing (Module 19), then package and deploy (Module 21).

6. Milestone / Deliverable Plan

Milestone	Expected Deliverable	Status
M1	Repository, architecture, environments and base project services	**Done**
M2	Authentication, three roles, backend APIs and database	**Done**
M3	Admin, Researcher and Hospital application foundations	**Done**
M4	Dataset inspection and preprocessing pipeline	**Done**
M5	Standalone local training across 8 architectures	**Done**
M6	Working multi-client Flower federation with FedAvg	**Done**
M7	Live monitoring and telemetry	**Partial**
M8	Resource-aware training (done) + FedProx experiments (not yet run)	**Partial**
M9	Differential Privacy integration	**Done**
M10	Byzantine detection (done) + TLS/security hardening (final checks open)	**In progress**
M11	Model versioning (partial) + local inference (done); global model delivery open	**Partial**
M12	Mobile app, communication, notifications: only hospital round notifications exist	**Partial**
M13	Full testing and research evaluation	**Partial**
M14	Packaged final software + documentation + demonstration	**Planned**

7. Core Workflows

Training requests and federation jobs are separate today: a request does not create a job.

A. Researcher training request (local training)

1. Researcher publishes a training request.
2. Hospital Operators choose to participate, or later withdraw.
3. Each hospital prepares its data and trains locally, linking the run to the request.
4. Hospitals report progress only, with no raw data; the Researcher follows it live.

B. Federated training (Admin-run job)

1. Admin creates a federation job: architecture, classes, round plan, strategy, DP and security settings.
2. Backend validates the configuration and records the job.
3. Hospital Operators send a join request with a local eligibility summary, and the Admin approves or rejects it.
4. Admin starts the federation; the backend starts one Flower server for the job.
5. Approved hospitals with the app open connect and authenticate with their login token (over TLS when enabled).
6. The current global model is distributed, and each hospital verifies its checksum.
7. Each hospital loads its private dataset, already inspected and preprocessed locally.
8. The hospital trains locally with the job's settings on its own data, applying DP when enabled.
9. The hospital sends a model update and required metadata over the Flower connection (TLS when enabled).
10. The server validates each update and runs the Module 12 security checks.
11. Accepted updates are aggregated with the job's strategy; a new global model version is stored.

12. Hospitals calibrate BatchNorm statistics on their own images and send only the statistics (non-DP Flower jobs).
13. Round status appears live on Admin and Hospital screens, and hospitals are notified.
14. Steps 6 to 13 repeat until the planned rounds finish or the stopping rule applies.
15. Admin evaluates a global model on a held-out set and may promote it to MAIN.
16. **Not built yet:** hospitals download the promoted model and use it for local inference.

8. Research Evaluation Matrix

Research Question	Comparison	Status
Which model suits the task and hardware?	8 architectures on one test set	**Done**
Does federation retain useful performance?	Centralized baseline vs Federated baseline	**Indicative only**
How does Non-IID data affect training?	Different client data distributions	**Tool ready**
Does FedProx help heterogeneous clients?	FedAvg vs FedProx	**Tool ready**
What is the cost of privacy?	Without DP vs With DP	**Indicative only**
Does security defense help?	Attack without defense vs attack with defense	**Partial (synthetic)**
Does resource adaptation help?	Fixed configuration vs adaptive configuration	**Planned**
What is the communication cost?	Bytes, rounds, duration and client participation	**Partial (bench data)**

9. Future-Extension Design

Modular interfaces allow additions without redesigning the platform. Possible future extensions include additional models, personalized federated learning, secure aggregation, Krum and adaptive Byzantine defenses, mutual TLS, patient-level privacy, a membership-inference audit, explainability, model compression, client dropout handling, DICOM and other medical modalities, cloud deployment, and true peer-to-peer/gossip federation.

10. Scope and Implementation Safeguards

- The working demonstration uses image classification on the public NIH malaria dataset; skin cancer and ISIC 2019 were also used in local runs.
- Hospitals are simulated on one machine with public datasets. This does not represent clinical deployment.
- Do not claim that one trained model supports every disease or medical modality.
- Do not describe the project as collecting trained patient data. Raw patient images remain local.
- Keep the global model concept separate from the backend itself: the backend is the software infrastructure; the global model is the versioned set of learned parameters produced by federation.
- Treat accuracy, privacy budget, scalability, communication efficiency and security effectiveness as experimental results, not guaranteed values.
- Keep all configurable values outside core source code.
- Document every experiment with dataset partitioning, model version, training settings and random seed.

11. Final Definition of Done

Requirement	Status
Admin can log in, approve users and view hospitals; Researcher can log in and publish training requests.	**Done**
Hospital can run the local client without exposing its raw medical dataset.	**Done (from source)**
Hospital can inspect and preprocess its local dataset.	**Done**
Multiple hospital clients can participate in federated training.	**Done (one machine)**
PyTorch training works locally.	**Done**
Flower federation and FedAvg work correctly.	**Done**
Non-IID experiments can be reproduced.	**Tool ready**
Resource-aware training works on heterogeneous client hardware.	**Partial (simulated)**
Differential Privacy can be enabled and evaluated.	**Done**
Secure communication and authentication are implemented.	**In progress**
Suspicious/malicious updates can be detected and handled.	**Done (known limits)**
Training metrics and security/resource telemetry appear in real time.	**Partial**
Model checkpoints are versioned and recoverable.	**Partial (no rollback)**
An approved global model can be deployed for local inference.	**Not done**
Desktop and mobile applications provide the required user-facing functionality.	**Partial (no mobile)**
Complete integration, security, performance and research experiments are documented.	**Partial**
The final software can be installed and demonstrated end-to-end.	**Not done**